

Rapport technique

Data Lake financier

Groupe : Artemiy Smogunov, Nathan Smadja-Tubiana, Patrice Ignongui

Projet EFREI 2025-2026

Rapport technique

Projet et données utilisées

Nous avons construit un data lake financier qui traite des cours journaliers. Le dépôt contient une source fichier et une source API.

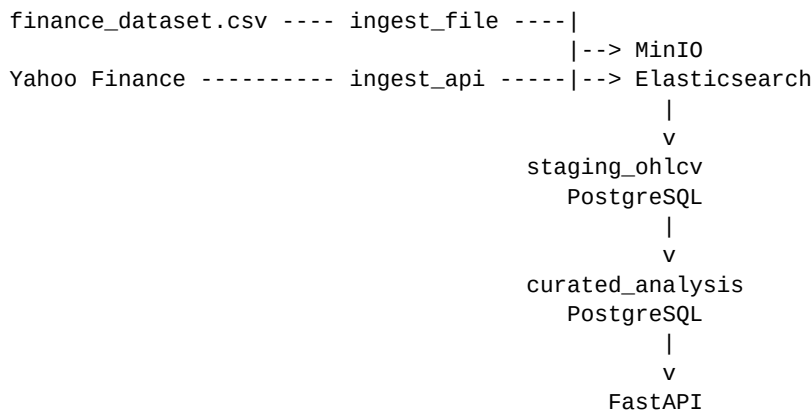
La source fichier est `data/finance_dataset.csv`. Elle contient l'historique AAPL du 3 janvier 2022 au 19 novembre 2024. Le pipeline la lit directement depuis le dépôt.

La seconde source est Yahoo Finance. Le code l'interroge avec `yfinance` pour les tickers définis dans `config/settings.py` : AAPL, MSFT, GOOGL, AMZN, NVDA, META, TSLA, BRK-B, JPM, JNJ, ^GSPC, ^DJI, ^IXIC et ^RUT.

Nous avons lancé la stack Docker et exécuté le DAG avant de rédiger ce rapport. Les nombres et comportements présentés ici viennent de cette exécution et du code présent dans le dépôt.

Architecture exécutée

Docker Compose lance PostgreSQL, MinIO, Elasticsearch, Airflow et FastAPI sur le même réseau `datalake-net`.



MinIO et Elasticsearch appartiennent tous les deux à la zone Raw, mais ils n'ont pas le même rôle. MinIO conserve les objets CSV et JSON. Elasticsearch conserve une ligne indexée par source, ticker et date afin que l'API puisse filtrer les données brutes.

PostgreSQL contient les tables `staging_ohlcv`, `curated_analysis` et `ingestion_logs`. Airflow utilise la même instance PostgreSQL pour ses propres tables.

Zone Raw

Ingestion du fichier

`ingestion/ingest_file.py` charge `data/finance_dataset.csv`, normalise les noms de colonnes et vérifie la présence de `ticker`, `date`, `open`, `high`, `low`, `close` et `volume`.

Le module :

1. convertit le ticker en majuscules ;
2. convertit les dates au format YYYY-MM-DD ;
3. convertit les colonnes numériques ;
4. retire les lignes sans ticker, date ou clôture ;
5. garde la dernière ligne en cas de doublon (ticker, date) ;
6. écrit un CSV par ticker dans le bucket raw-financial-data ;
7. indexe chaque ligne dans raw_financial_events.

L'identifiant Elasticsearch suit le format source_ticker_date. Une nouvelle ingestion de la même source, du même ticker et de la même date met donc à jour le document au lieu d'en créer un second.

Ingestion Yahoo Finance

ingestion/ingest_api.py récupère les dernières dates disponibles. Le payload JSON contient le ticker, la date de récupération, la source, les métadonnées disponibles et les enregistrements OHLCV.

Le JSON complet est écrit dans raw-api-data. Les cours sont ensuite indexés dans Elasticsearch avec la source yfinance_api.

Le DAG demande deux jours de recul. L'endpoint manuel accepte une période comme 5d ou 1mo.

Zone Staging

transformation/staging/transform_staging.py relit toutes les lignes Raw d'un ticker depuis Elasticsearch. Il les trie par date, les déduplique et calcule les indicateurs sur la série obtenue.

Les colonnes ajoutées sont :

- sma_20 et sma_50 ;
- ema_12 et ema_26 ;
- rsi_14 ;
- macd et macd_signal ;
- bollinger_upper et bollinger_lower ;
- daily_return ;
- volatility_20.

Les valeurs non numériques, infinies ou absentes sont converties en NULL avant l'écriture PostgreSQL. L'upsert utilise la contrainte unique (ticker, date).

Zone Curated

transformation/curated/transform_curated.py lit les données Staging par ticker.

Détection d'anomalies

Le modèle Isolation Forest travaille sur quatre colonnes :

- rendement journalier ;
- volatilité sur 20 périodes ;
- z-score glissant du volume ;
- RSI 14.

Les colonnes manquantes sont remplacées par leur médiane. Le modèle utilise 100 arbres, `random_state=42` et une contamination de 5 %. Il ne se lance pas si le ticker contient moins de 30 lignes. Dans ce cas, le score vaut 0 et aucune ligne n'est marquée comme anomalie.

Après la détection, le code attribue un type selon des seuils fixes :

- `flash_crash` sous -5 % de rendement journalier ;
- `price_spike` au-dessus de 5 % ;
- `volume_spike` si le z-score absolu du volume dépasse 3 ;
- `high_volatility` si la volatilité dépasse 4 % ;
- `unknown_anomaly` pour les autres points isolés.

Tendance et signal

La tendance est `bullish` si le cours est supérieur à la SMA 20, elle-même supérieure à la SMA 50. La condition inverse produit `bearish`. Les autres cas donnent `neutral`.

Le signal vaut `buy` lorsque le RSI est inférieur à 30 et que le MACD dépasse sa ligne de signal. Il vaut `sell` lorsque le RSI dépasse 70 et que le MACD est sous sa ligne de signal. Les autres cas donnent `hold`.

Nous utilisons ces valeurs pour montrer l'enrichissement Curated. Elles ne constituent pas des conseils financiers.

Orchestration Airflow

Le DAG `financial_data_lake_pipeline` suit ce graphe :

```
graph LR
    start --> ingest_file[ingest_file]
    ingest_file --> ingest_api[ingest_api]
    ingest_file --> transform_staging[transform_staging]
    ingest_api --> transform_staging
    transform_staging --> transform_curated[transform_curated]
    transform_curated --> log_summary[log_summary]
    log_summary --> end
```

Il est planifié avec l'expression `0 6 * * 1-5`, soit 6 h UTC du lundi au vendredi. Il n'exécute pas de rattrapage historique et limite les exécutions actives à une. Chaque tâche Python peut être retentée deux fois avec cinq minutes d'attente.

Les deux ingestions démarrent ensemble. Elles transmettent leurs tickers réussis par XCom. Staging traite l'union de ces tickers. Curated ne traite que les tickers réussis dans Staging.

 Airflow

DAGs

Cluster Activity

Datasets

Security

Browse

Admin

Docs

17:24 UTC

AU

DAGs

All 1

Active 1

Paused 0

Running 0

Failed 0

Filter DAGs by tag

Search DAGs

Auto-refresh

DAG	Owner	Runs	Schedule	Last Run	Next Run	Recent Tasks	Actions	Li
financial_data_lake_pipeline data-lake finance yfinance data-engineer		3	0 6 ** 1-5	2026-07-10, 17:17:03	2026-07-10, 06:00:00	7		

«

<

1

>

»

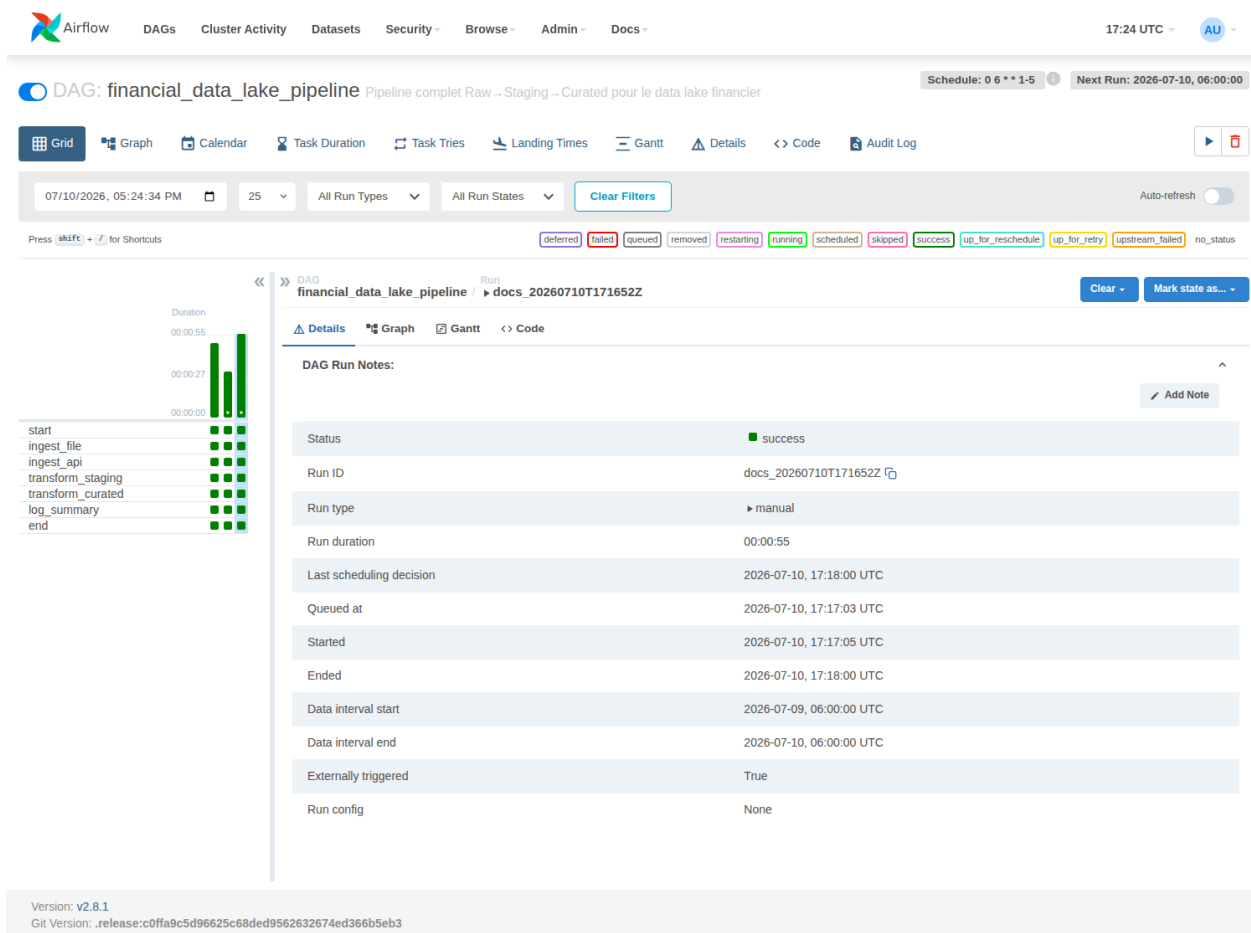
Showing 1-1 of 1 DAGs

Version: v2.8.1

Git Version: .release:c0ffa9c5d96625c68ded9562632674ed366b5eb3

Liste du DAG dans Airflow

La page d'accueil Airflow montre un DAG actif, aucun DAG en échec et sept tâches récentes en succès. Le planning visible correspond à la configuration du code.



Détail de l'exécution Airflow

Nous avons déclenché le run docs_20260710T171652Z. Son état final est success. Il a commencé à 17:17:05 UTC et s'est terminé à 17:18:00 UTC, soit 55 secondes. La grille montre les sept tâches en vert.

Les états relevés en ligne de commande confirment la capture :

start	success
ingest_file	success
ingest_api	success
transform_staging	success
transform_curated	success
log_summary	success
end	success

API FastAPI

FastAPI expose les données sans accès direct aux bases.

Financial Data Lake API 1.0.0 OAS 3.1

/openapi.json

API Gateway du data lake financier (Yahoo Finance / yfinance). Expose les trois zones du data lake (Raw, Staging, Curated) et fournit des endpoints d'ingestion standard et optimisé.

Health		^
GET	/health	Vérification de l'état des services
Stats		^
GET	/stats	Métriques de remplissage du data lake
Raw		^
GET	/raw	Données brutes de la zone Raw
GET	/raw/objects	Liste des objets MinIO dans la zone Raw
Staging		^
GET	/staging	Données staging avec indicateurs techniques
GET	/staging/tickers	Liste des tickers disponibles en staging
Curated		^
GET	/curated	Données curated avec scores d'anomalie
GET	/curated/anomalies/summary	Résumé des anomalies par ticker
GET	/curated/signals	Tickers avec signaux actifs (buy/sell)
Ingest		^
POST	/ingest	Ingestion synchrone avec benchmark
Ingest Fast		^
POST	/ingest_fast	Ingestion optimisée
Schemas		^
HTTPValidationError > Expand all object		
HealthResponse > Expand all object		
IngestFastRequest > Expand all object		
IngestFastResponse > Expand all object		
IngestRequest > Expand all object		
IngestResponse > Expand all object		
ServiceStatus > Expand all object		
ValidationError > Expand all object		

Routes Swagger

La capture montre les groupes Health, Stats, Raw, Staging, Curated, Ingest et Ingest Fast. Ce sont les routeurs enregistrés dans `api/main.py`.

Lecture des zones

GET `/raw` interroge Elasticsearch. Les paramètres permettent de filtrer le ticker, la source, la date de début, la date de fin et le nombre de documents.

GET `/raw/objects` liste les objets du bucket fichier ou API dans MinIO.

GET `/staging` lit PostgreSQL avec les filtres ticker et dates, puis applique `limit` et `offset`.

GET `/curated` ajoute les filtres `anomalies_only` et `signal`.

Les routes `/staging/tickers`, `/curated/anomalies/summary` et `/curated/signals` fournissent des regroupements déjà calculés par PostgreSQL.

Déclenchement manuel

POST /ingest traite les tickers l'un après l'autre. Le téléchargement, l'écriture Raw, Staging et Curated sont mesurés séparément.

POST /ingest_fast utilise un pool de huit threads pour télécharger les tickers et envoyer les fichiers dans MinIO. Il regroupe les documents dans une seule écriture Elasticsearch et les lignes Staging avec execute_values.

Les deux routes limitent les lots à 200 tickers et renvoient les erreurs par étape.

Résultats observés

Après le lancement de Docker Compose et l'exécution du DAG, GET /health a retourné :

overall	ok
PostgreSQL	ok
MinIO	ok
Elasticsearch	ok, version 8.11.0

GET /stats a retourné :

Objets MinIO fichier	200
Objets MinIO API	42
Documents Elasticsearch	1763
Lignes Staging	1243
Lignes Curated	1243
Anomalies	37

Résultats de l'exécution

10 juillet 2026. Données lues après le run Airflow docs_20260710T171652Z.

Etat des services

PostgreSQL ok	MinIO ok	Elasticsearch ok version 8.11.0
-------------------------	--------------------	--

Volumes observés

Objets MinIO 242 200 fichier, 42 API	Documents Raw ES 1763	Lignes Staging 1243
Lignes Curated 1243	Anomalies Curated 37	Tâches Airflow réussies 7 / 7

Lecture

- | Raw conserve les objets et les documents reçus.
- | Staging et Curated contiennent le même nombre de lignes car Curated enrichit chaque ligne Staging.
- | Les 37 anomalies viennent du modèle Isolation Forest exécuté par ticker.

Résumé des résultats

La capture regroupe l'état des services et les volumes observés. MinIO contient plus d'objets que le nombre de tickers parce que l'ingestion API crée un objet horodaté à chaque exécution. Elasticsearch utilise des identifiants stables et met à jour une même date au lieu de dupliquer chaque passage.

Staging et Curated ont tous les deux 1 243 lignes. Curated enrichit les lignes Staging et conserve la même clé (`ticker`, `date`). Les 37 anomalies représentent environ 3 % du total global. Le taux n'est pas exactement 5 % au niveau global parce que le modèle est entraîné séparément par ticker et ne s'exécute pas sous 30 lignes.

Lecture d'une ligne AAPL

Passage des données dans les trois zones

Exemple : AAPL, dernière date disponible.

Raw		Staging		Curated	
Date	2026-07-10	Date	2026-07-10	Date	2026-07-10
Clôture	314.5398864746094	Clôture	314.539886	Score anomalie	-0.527124
Volume	14262307	SMA 20	313.493982	Anomalie	False
Source	yfinance_fast	MACD	0.334471	Tendance	neutral
Date d'ingestion	2026-07-10T17:09:55.683990+00:00	Variation	-0.005313	Signal	hold
		Volatilité	0.008536	Traitement	2026-07-10 17:10:06.641214

Raw conserve la donnée reçue et sa provenance.

Staging ajoute les indicateurs techniques.

Curated produit un score d'anomalie, une tendance et un signal.

AAPL dans Raw, Staging et Curated

La ligne présentée porte sur le 10 juillet 2026.

Dans Raw, le cours de clôture vaut 314,539886 et le volume 14 262 307. La source est yfinance_fast. La zone garde aussi l'heure d'ingestion.

Dans Staging, le cours est arrondi par le type PostgreSQL. La SMA 20 vaut 313,493982, le MACD 0,334471, le rendement journalier -0,005313 et la volatilité 0,008536.

Dans Curated, le score d'anomalie vaut -0,527124. La ligne n'est pas classée comme anomalie. Sa tendance est neutra l et son signal est ho ld.

Cette capture montre que chaque zone ajoute des informations sans changer le ticker ni la date.

Benchmark

Le benchmark commité dans livrables/benchmark_ingest_vs_ingest_fast . j son utilise trois répétitions et alterne l'ordre des deux endpoints.

Pour un ticker :

```
/ingest      1 073,29 ms
/ingest_fast  989,86 ms
Gain         7,77 %
```

Pour 100 tickers :

/ingest	104 825,65 ms
/ingest_fast	12 230,36 ms
Gain	88,33 %

Les douze appels du benchmark ont retourné succès. Sur un ticker, le coût de création des threads et les étapes communes limitent le gain. Sur 100 tickers, les téléchargements parallèles et les écritures groupées réduisent nettement la durée.

Le JSON associe ces mesures au commit 85aa820. Une modification des pipelines demande un nouveau benchmark avant de remplacer ces chiffres.

Tests exécutés

La commande utilisée est :

```
PYTHONPATH=.:api .venv/bin/python -m pytest -q
```

Résultat : 12 tests réussis.

Les tests vérifient notamment :

- le comportement du RSI sur une série en hausse et une série constante ;
- les règles de tendance et de signal ;
- la déduplication Staging ;
- la stabilité des identifiants Elasticsearch ;
- la validation du CSV ;
- le calcul du gain et du statut du benchmark.

Nous avons aussi compilé les modules Python, validé docker compose config et vérifié l'absence d'erreurs d'import Airflow.

Limites constatées pendant l'exécution

Le service Yahoo Finance reste externe au projet. Une requête peut prendre plus de temps ou ne retourner aucune ligne.

Le RSI demande quatorze observations. Dans /ingest_fast, les indicateurs Staging sont calculés sur le lot téléchargé avant l'upsert. Avec period: 5d, le RSI des nouvelles lignes reste donc vide. Pour notre capture courte, le signal Curated reste hold quand le RSI manque.

Curated relit l'historique Staging complet des tickers demandés. Lors d'une petite ingestion, son compteur processed peut donc être bien supérieur au nombre de nouvelles lignes téléchargées.

Le paramètre de contamination fixe une proportion attendue d'anomalies. Une anomalie indique un point isolé par rapport aux variables utilisées, pas une erreur certaine dans le marché.

Les comptes et mots de passe définis dans Docker Compose servent uniquement au lancement local du projet.

Reproduction

```
docker compose up -d
curl http://localhost:8000/health
curl http://localhost:8000/stats
```

Déclenchement du DAG :

```
docker compose exec airflow-scheduler \
    airflow dags trigger financial_data_lake_pipeline
```

Test d'une ingestion manuelle :

```
curl -X POST http://localhost:8000/ingest_fast \
-H "Content-Type: application/json" \
-d '{"data":{"tickers":["AAPL","MSFT","NVDA"],"period":"5d","run_staging":true,"run_curated":t
```

Contrôle des trois zones :

```
curl "http://localhost:8000/raw?ticker=AAPL&limit=1"
curl "http://localhost:8000/staging?ticker=AAPL&limit=1"
curl "http://localhost:8000/curated?ticker=AAPL&limit=1"
```